



EAST WEST UNIVERSITY

Assignment

Submitted to:

- ***Dr. Md. Manowarul Islam***
- ***Adjunct faculty***
- ***Department of Computer Science and Technology***
- ***East West University***

Submitted by:

- ***Name*** : ***Rakib Al Hasan***
- ***Student ID*** : ***2024-1-60-143***
- ***Course Title*** : ***Data Structure and Algorithm***
- ***Course Code*** : ***CSE 207***
- ***Section*** : ***10***
- ***Semester*** : ***Summer 2025***
- ***Date of Submission*** : ***20.08.2025***

1. Write a program in C to implement a stack that only allows even numbers to be pushed. Odd numbers should be rejected with an appropriate message.

Sample Input	Sample Output
Push 20	Stack Contain: 20
Push 23	Cannot be Inserted
Push 40	Stack Contain: 20 40

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

int stack[MAX], top = -1;

void push(int x)
{
    if (x % 2 != 0)
    {
        printf("\t%d Cannot be Inserted (only even numbers allowed)!!\n", x);
        return;
    }
    if (top == MAX - 1)
    {
        printf("\tStack Overflow!!\n\tPush operation failed!!");
        return;
    }
    stack[++top] = x;
    printf("\t%d pushed to stack.\n", x);
}

void pop()
{
    if (top == -1)
    {
        printf("\tStack Underflow (empty)!!\n\tPop operation failed!!\n");
        return;
    }
}
```

```

    }
    printf("\t%d popped from stack.\n", stack[top--]);
}

void display()
{
    if (top == -1)
    {
        printf("\tStack is Empty!!\n");
        return;
    }
    printf("\tStack Contains : ");
    for (int i = 0; i <= top; i++)
    {
        printf("\t%d ", stack[i]);
    }
    printf("\n");
}

int main()
{
    int choice, item;
    //char cont = 'y';
    printf("-----MENU-----\n");
    printf("\tWhat type of operation do you want to perform?\n");
    printf("\t1. Push\n\t2. Pop\n\t3. Display\n\t4. Exit\n\t");

    while (1)
    {
        //system("cls");
        printf("\n\tEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("\t> Your choice: Push operation\n");
                printf("\t> Enter a number to push into the stack: ");
                scanf("%d", &item);
                push(item);
                break;

            case 2:

```

```

        printf("\t> Your choice: Pop operation\n");
        pop();
        break;

    case 3:
        printf("\t> Your choice: Display operation\n");
        display();
        break;

    case 4:
        printf("\tBYE!!!\n");
        exit(0);

    default:
        printf("\t>Invalid Input!!\n");
}

//printf("\n\tDo you want to continue? (y for yes/any other key for no) : ");
//scanf(" %c", &cont);
}

printf("\n\tProgram Terminated. Goodbye!!\n");
return 0;
}

```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Untitled1.exe"

```
-----MENU-----
What type of operation do you want to perform?
1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1
> Your choice: Push operation
> Enter a number to push into the stack: 20
20 pushed to stack.

Enter your choice: 1
> Your choice: Push operation
> Enter a number to push into the stack: 23
23 Cannot be Inserted (only even numbers allowed)!!

Enter your choice: 1
> Your choice: Push operation
> Enter a number to push into the stack: 40
40 pushed to stack.

Enter your choice: 3
> Your choice: Display operation
Stack Contains :      20      40

Enter your choice: 4
BYE!!!

Process returned 0 (0x0)   execution time : 21.757 s
Press any key to continue.
```

2. Write a program in C to Create a stack with a constraint that the **sum of all elements** in the stack should never exceed **100**. Prevent further pushes if adding the new element exceeds this limit.

Sample Input & Output
<i>If $current_sum + new_element > 100$, reject the push and display: "Sum limit exceeded."</i>

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

int stack[MAX], top = -1, sum = 0;

void push(int x)
{
    if (sum + x > 100)
    {
        printf("\tSum limit exceeded. Cannot insert %d (Current Sum = %d)\n", x, sum);
        return;
    }
    if (top == MAX - 1)
    {
        printf("\tStack Overflow!!\n\tPush operation failed!!");
        return;
    }
    stack[++top] = x;
    sum += x;
    printf("\t%d pushed to stack. (Current Sum = %d)\n", x, sum);
}

void pop()
{
    if (top == -1)
    {
        printf("\tStack Underflow (empty)!!\n\tPop operation failed!!\n");
        return;
    }
}
```

```

    int removed = stack[top--];
    sum -= removed;
    printf("\tPopped element : %d (Current Sum = %d)\n", removed, sum);
}

```

```

void display()
{
    if (top == -1)
    {
        printf("\tStack is Empty!!\n");
        return;
    }
    printf("\tStack Contains : ");
    for (int i = 0; i <= top; i++)
    {
        printf("%d ", stack[i]);
    }
    printf("\n (Sum = %d)\n", sum);
}

```

```

int main()
{
    int choice, item;
    //char cont = 'y';
    printf("-----MENU-----\n");
    printf("\tWhat type of operation do you want to perform?\n");
    printf("\t1. Push\n\t2. Pop\n\t3. Display\n\t4. Exit\n\t");

```

```

while (1)
{
    //system("cls");
    printf("\n\tEnter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            printf("\t> Your choice: Push operation\n");
            printf("\t> Enter a number to push into the stack: ");
            scanf("%d", &item);

```

```

        push(item);
        break;

    case 2:
        printf("\t> Your choice: Pop operation\n");
        pop();
        break;

    case 3:
        printf("\t> Your choice: Display operation\n");
        display();
        break;

    case 4:
        printf("\tBYE!!!\n");
        exit(0);

    default:
        printf("\t> Invalid Input!!\n");
}

//printf("\n\tDo you want to continue? (y for yes/any other key for no) : ");
//scanf(" %c", &cont);
}

printf("\n\tProgram Terminated. Goodbye!!\n");
return 0;
}

```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Untitled2.exe"

-----MENU-----

What type of operation do you want to perform?

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 15

15 pushed to stack. (Current Sum = 15)

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 33

33 pushed to stack. (Current Sum = 48)

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 41

41 pushed to stack. (Current Sum = 89)

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 0

0 pushed to stack. (Current Sum = 89)

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 23

Sum limit exceeded. Cannot insert 23 (Current Sum = 89)

Enter your choice: 3

> Your choice: Display operation

Stack Contains : 15 33 41 0 (Sum = 89)

Enter your choice: 4

BYE!!!

Process returned 0 (0x0) execution time : 41.235 s

Press any key to continue.

3. Write a program in C to Push N elements to the stack, then implement a function to reverse the **top K elements** without using any extra stack or array.

Sample Input	Sample Output
Push : 10 20 30 40 50 K=3	Reversed Stack : 10 20 50 40 30

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#define MAX 10
```

```
int stack[MAX], top = -1;
```

```
void push(int x)
```

```
{
```

```
    if (top == MAX - 1)
```

```
    {
```

```
        printf("\tStack Overflow!!\n\tPush operation failed!!");
```

```
        return;
```

```
    }
```

```
    stack[++top] = x;
```

```
    printf("\t%d pushed to stack.\n", x);
```

```
}
```

```
void pop()
```

```
{
```

```
    if (top == -1)
```

```
    {
```

```
        printf("\tStack Underflow (empty)!!\n\tPop operation failed!!\n");
```

```
        return;
```

```
    }
```

```
    printf("\t%d popped from stack.\n", stack[top--]);
```

```
}
```

```
void reverseTopK(int k)
```

```
{
```

```
    if (k > top + 1)
```

```
    {
```

```
        printf("\tNot enough elements to reverse!!\n");
```

```
        return;
```

```
    }
```

```
    int i = top - k + 1, j = top;
```

```

while (i < j)
{
    int temp = stack[i];
    stack[i] = stack[j];
    stack[j] = temp;
    i++;
    j--;
}
printf("\tTop %d elements reversed\n", k);
}

void display()
{
    if (top == -1)
    {
        printf("\tStack is Empty!!\n");
        return;
    }
    printf("\tStack Contains : ");
    for (int i = 0; i <= top; i++)
    {
        printf("\t%d ", stack[i]);
    }
    printf("\n");
}

int main()
{
    int choice, item, k;
    //char cont = 'y';
    printf("-----MENU-----\n");
    printf("\tWhat type of operation do you want to perform?\n");
    printf("\t1. Push\n\t2. Pop\n\t3. Display\n\t4. Reverse Top K\n\t5. Exit\n\t");

    while (1)
    {
        //system("cls");

        printf("\n\tEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {

```

```

case 1:
    printf("\t> Your choice: Push operation\n");
    printf("\t> Enter a number to push into the stack: ");
    scanf("%d", &item);
    push(item);
    break;

case 2:
    printf("\t> Your choice: Pop operation\n");
    pop();
    break;

case 3:
    printf("\t> Your choice: Display operation\n");
    display();
    break;

case 4:
    printf("\tEnter K : ");
    scanf("%d", &k);
    reverseTopK(k);
    break;

case 5:
    printf("\tBYE!!!\n");
    exit(0);

default:
    printf("\t>Invalid Input!!\n");
}

//printf("\n\tDo you want to continue? (y for yes/any other key for no) : ");
// scanf(" %c", &cont);
}

printf("\n\tProgram Terminated. Goodbye!!\n");
return 0;
}

```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Untitled3.exe"

-----MENU-----

What type of operation do you want to perform?

1. Push
2. Pop
3. Display
4. Reverse Top K
5. Exit

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 10
10 pushed to stack.

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 20
20 pushed to stack.

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 30
30 pushed to stack.

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 40
40 pushed to stack.

Enter your choice: 1

> Your choice: Push operation

> Enter a number to push into the stack: 50
50 pushed to stack.

Enter your choice: 3

> Your choice: Display operation

Stack Contains : 10 20 30 40 50

Enter your choice: 4

Enter K : 3

Top 3 elements reversed

Enter your choice: 3

> Your choice: Display operation

Stack Contains : 10 20 50 40 30

Enter your choice: 5

BYE!!!

Process returned 0 (0x0) execution time : 32.916 s

4. Write a program in C that implements a stack which **ignores duplicate values** during push. If an element already exists in the stack, it should not be added again.

Sample Input	Sample Output
Push 10	Element 10 pushed to stack.
Push 20	Element 20 pushed to stack.
Push 30	Element 30 pushed to stack.
Push 20	Duplicate entry ignored.
Push 40	Element 40 pushed to stack.
Push 10	Duplicate entry ignored.
Display ()	Stack Contain: 10 20 30 40

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100
```

```
int stack[MAX], top = -1;
```

```
int exists(int x)
{
    for (int i = 0; i <= top; i++)
    {
        if (stack[i] == x)
        {
            return 1;
        }
    }
    return 0;
}
```

```
void push(int x)
{
    if (exists(x))
    {
        printf("\t%d Cannot be Inserted (Duplicate entry ignored)!!\n" , x);
        return;
    }
}
```

```

    if (top == MAX - 1)
    {
        printf("\tStack Overflow!!\n\tPush operation failed!!");
        return;
    }
    stack[++top] = x;
    printf("\tElement %d pushed to stack.\n", x);
}

void pop()
{
    if (top == -1)
    {
        printf("\tStack Underflow (empty)!!\n\tPop operation failed!!\n");
        return;
    }
    printf("\tPopped element: %d\n", stack[top--]);
}

void display()
{
    if (top == -1)
    {
        printf("\tStack is Empty!!\n");
        return;
    }
    printf("\tStack Contains : ");
    for (int i = 0; i <= top; i++)
    {
        printf("\t%d ", stack[i]);
    }
    printf("\n");
}

int main()
{
    int choice, item;
    //char cont = 'y';
    printf("----- MENU ----- \n");
    printf("\tWhat type of operation do you want to perform?\n");
    printf("\t1. Push\n\t2. Pop\n\t3. Display\n\t4. Exit\n");

    while (1)

```

```

{
    //system("cls");

    printf("\tEnter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            printf("\tYour choice: Push operation\n");
            printf("\tEnter a number to push into the stack: ");
            scanf("%d", &item);
            push(item);
            break;

        case 2:
            printf("\tYour choice: Pop operation\n");
            pop();
            break;

        case 3:
            printf("\tYour choice: Display operation\n");
            display();
            break;

        case 4:
            printf("\tBYE!!!\n");
            exit(0);

        default:
            printf("\tInvalid Input!!\n");
    }

    //printf("\n\tDo you want to continue? (y for yes/any other key for
no): ");
    //scanf(" %c", &cont);
}

printf("\n\tProgram Terminated. Goodbye!!\n");
return 0;
}

```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Untitled4.exe"

----- MENU -----

What type of operation do you want to perform?

1. Push
2. Pop
3. Display
4. Exit

Enter your choice: 1

Your choice: Push operation

Enter a number to push into the stack: 10

Element 10 pushed to stack.

Enter your choice: 1

Your choice: Push operation

Enter a number to push into the stack: 20

Element 20 pushed to stack.

Enter your choice: 1

Your choice: Push operation

Enter a number to push into the stack: 30

Element 30 pushed to stack.

Enter your choice: 1

Your choice: Push operation

Enter a number to push into the stack: 20

20 Cannot be Inserted (Duplicate entry ignored)!!

Enter your choice: 1

Your choice: Push operation

Enter a number to push into the stack: 40

Element 40 pushed to stack.

Enter your choice: 1

Your choice: Push operation

Enter a number to push into the stack: 10

10 Cannot be Inserted (Duplicate entry ignored)!!

Enter your choice: 3

Your choice: Display operation

Stack Contains : 10 20 30 40

Enter your choice: 4

BYE!!!

Process returned 0 (0x0) execution time : 33.673 s

Press any key to continue.

5. Write a program in C where in addition to normal peek (top), you can also **peek the middle element** of the stack.

Sample Input	Sample Output
Push 10	Element 10 pushed to stack.
Push 20	Element 20 pushed to stack.
Push 30	Element 30 pushed to stack.
Display ()	Stack Contain: 10 20 30
Special peek ()	Middle element is 20.

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 10

int stack[MAX], top = -1;

void push(int x)
{
    if (top == MAX - 1)
    {
        printf("\tStack Overflow!!\n\tPush operation failed!!");
        return;
    }
    stack[++top] = x;
    printf("\tElement %d pushed to stack.\n", x);
}

void pop()
{
    if (top == -1)
    {
        printf("\tStack Underflow (empty)!!\n\tPop operation failed!!\n");
        return;
    }
    printf("\tPopped element: %d\n", stack[top--]);
}
```

```

void display()
{
    if (top == -1)
    {
        printf("\tStack is Empty!!\n");
        return;
    }
    printf("\tStack Contains : ");
    for (int i = 0; i <= top; i++)
    {
        printf("\t%d ", stack[i]);
    }
    printf("\n");
}

void middlePeek()
{
    if (top == -1)
    {
        printf("\tStack Empty\n");
        return;
    }
    int mid = top / 2; // Middle index
    printf("\tMiddle element is %d.\n", stack[mid]);
}

void peek()
{
    if (top == -1)
    {
        printf("\tStack Empty\n");
        return;
    }
    printf("\tTop element is %d.\n", stack[top]);
}

int main()
{
    int choice, item;
    //char cont = 'y';
    printf("-----MENU-----\n");
    printf("\tWhat type of operation do you want to perform?\n");

```

```
printf("\t1. Push\n\t2. Pop\n\t3. Display\n\t4. Peek (Top)\n\t5. Special  
Peek (Middle)\n\t6. Exit\n");
```

```
while (1)
{
    //system("cls");

    printf("\n\tEnter your choice : ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            printf("\tYour choice : Push operation\n");
            printf("\tEnter a number to push into the stack : ");
            scanf("%d", &item);
            push(item);
            break;

        case 2:
            printf("\tYour choice: Pop operation\n");
            pop();
            break;

        case 3:
            printf("\tYour choice: Display operation\n");
            display();
            break;

        case 4:
            peek();
            break;

        case 5:
            middlePeek();
            break;

        case 6:
            printf("\tBYE!!!\n");
            exit(0);

        default:
            printf("\t> Invalid Input!!\n");
    }
}
```

```

    }

    //printf("\n\tDo you want to continue? (y for yes/any other key for no) :
");
    //scanf(" %c", &cont);
}

printf("\n\tProgram Terminated. Goodbye!!\n");
return 0;
}

```

```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Untitled5.exe"
-----MENU-----
What type of operation do you want to perform?
1. Push
2. Pop
3. Display
4. Peek (Top)
5. Special Peek (Middle)
6. Exit

Enter your choice : 1
Your choice : Push operation
Enter a number to push into the stack : 10
Element 10 pushed to stack.

Enter your choice : 1
Your choice : Push operation
Enter a number to push into the stack : 20
Element 20 pushed to stack.

Enter your choice : 1
Your choice : Push operation
Enter a number to push into the stack : 30
Element 30 pushed to stack.

Enter your choice : 5
Middle element is 20.

Enter your choice : 3
Your choice: Display operation
Stack Contains :      10      20      30

Enter your choice : 6
BYE!!!

Process returned 0 (0x0)   execution time : 34.412 s
Press any key to continue.

```

6. Write a program in C to implement a queue that only allows prime numbers to be enqueued. Non-prime numbers should be rejected with an appropriate message.

Sample Input	Sample Output
Enqueue 7	Element 7 inserted to Queue
Enqueue 10	10 is not a prime Number
Enqueue 13	Element 13 inserted to Queue.
Display()	Queue Contains: 7 13

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>

#define MAX 10

int queue[MAX], front = -1, rear = -1;

int isPrime(int n)
{
    if (n < 2)
    {
        return 0;
    }
    for (int i = 2; i <= sqrt(n); i++)
    {
        if (n % i == 0)
        {
            return 0;
        }
    }
    return 1;
}

void enqueue(int x)
{
    if (!isPrime(x))
```

```

{
    printf("\t%d is not a prime Number\n", x);
    return;
}
if (rear == MAX - 1)
{
    printf("\tQueue Overflow!!\n\nEnqueue operation failed!!");
    return;
}
if (front == -1) front = 0;
{
    queue[++rear] = x;
}
printf("\tElement %d inserted to Queue\n", x);
}

void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("\tQueue Underflow!!\n\nDequeue operation failed!!");
        return;
    }
    printf("\tDequeued element: %d\n", queue[front++]);
}

void display()
{
    if (front == -1 || front > rear)
    {
        printf("\tQueue Empty!!\n");
        return;
    }
    printf("\tQueue Contains : ");
    for (int i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

```

```

int main()
{
    int choice, item;
    //char cont = 'y';
    printf("----- MENU -----\\n");

    printf("\\t1. Enqueue\\n\\t2. Dequeue\\n\\t3. Display\\n\\t4. Exit\\n");
    printf("\\tWhat type of operation do you want to perform?\\n");

    while (1)
    {
        //system("cls");

        printf("\\tEnter your choice : ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("\\tYour choice : Enqueue operation\\n");
                printf("\\tEnter a number to enqueue : ");
                scanf("%d", &item);
                enqueue(item);
                break;

            case 2:
                printf("\\tYour choice : Dequeue operation\\n");
                dequeue();
                break;

            case 3:
                printf("\\tYour choice: Display operation\\n");
                display();
                break;

            case 4:
                printf("\\tBYE!!\\n");
                exit(0);

            default:
                printf("\\tInvalid Input!!\\n");

```

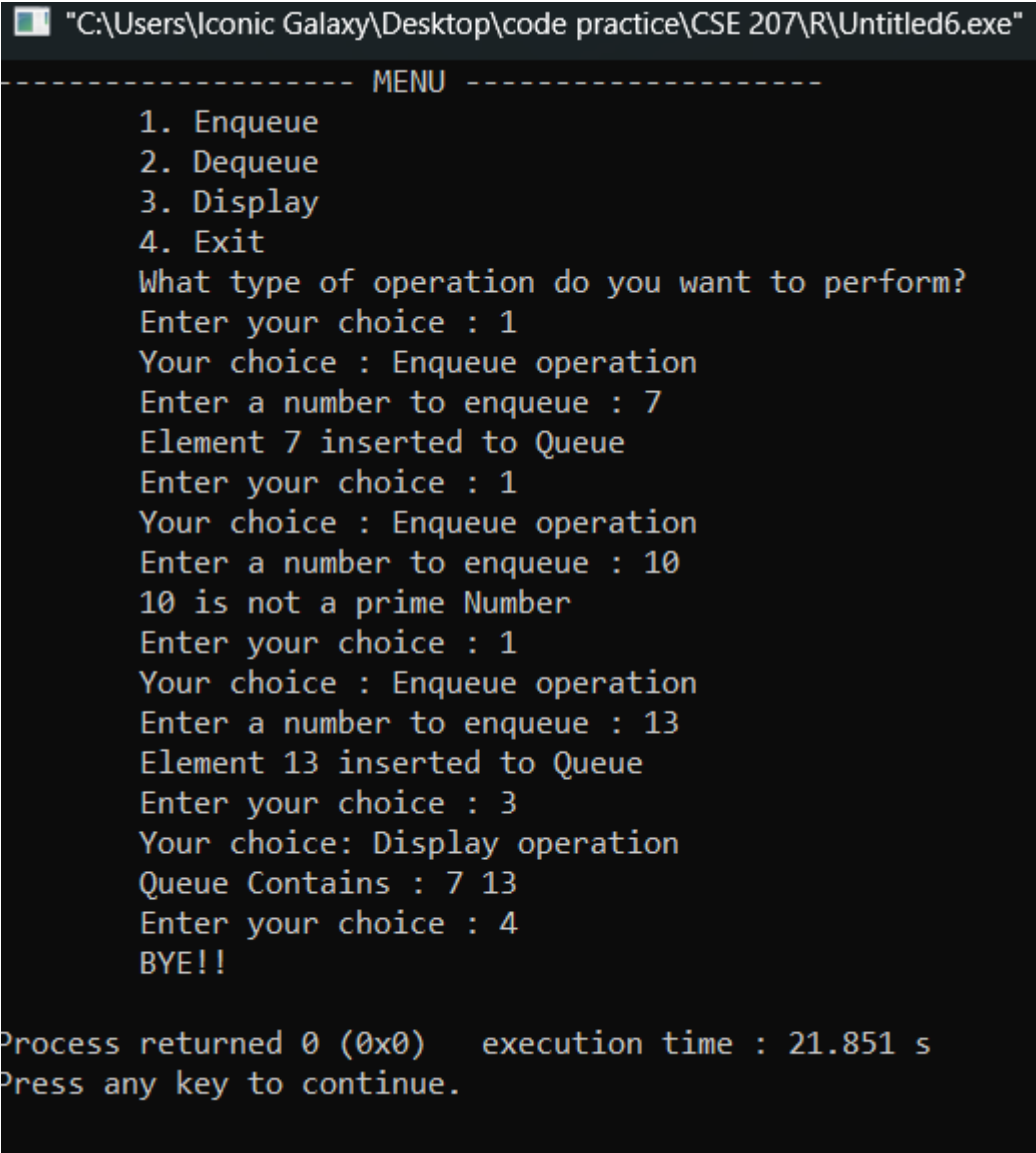
```

    }

    //printf("\n\tDo you want to continue? (y for yes/any other key for no) : ");
    //scanf(" %c", &cont);
}

printf("\n\tProgram Terminated. Goodbye!!\n");
return 0;
}

```



```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Untitled6.exe"
----- MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
What type of operation do you want to perform?
Enter your choice : 1
Your choice : Enqueue operation
Enter a number to enqueue : 7
Element 7 inserted to Queue
Enter your choice : 1
Your choice : Enqueue operation
Enter a number to enqueue : 10
10 is not a prime Number
Enter your choice : 1
Your choice : Enqueue operation
Enter a number to enqueue : 13
Element 13 inserted to Queue
Enter your choice : 3
Your choice: Display operation
Queue Contains : 7 13
Enter your choice : 4
BYE!!

Process returned 0 (0x0)   execution time : 21.851 s
Press any key to continue.

```

7. Write a program in C to implement a queue where the sum of all elements should never exceed 50. Reject new elements that would cause the sum to exceed 50.

Sample Input	Sample Output
Enqueue 20	Element 20 inserted to Queue
Enqueue 15	Element 15 inserted to Queue.
Enqueue 18	Inserting 18 would exceed sum limit (current sum = 35). Rejected
Enqueue 10	Element 10 inserted to Queue.
Display ()	Queue Contains: 20 15 10

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

int queue[MAX], front = -1, rear = -1, sum = 0;

void enqueue(int x)
{
    if (sum + x > 50)
    {
        printf("\tSum limit exceeded. Cannot insert %d (Current Sum = %d)\n", x, sum);
        return;
    }
    if (rear == MAX - 1)
    {
        printf("\tQueue Overflow!!\n\nEnqueue operation failed!!");
        return;
    }
}
```

```

    if (front == -1) front = 0;
    {
        queue[++rear] = x;
        sum += x;
        printf("\tElement %d inserted to Queue.(Current Sum = %d)\n", x,
sum);
    }
}

```

```

void dequeue()
{
    if (front == -1 || front > rear)
    {
        printf("\tQueue Underflow!!\n\tDequeue operation failed!!");
        return;
    }
    int removed = queue[front++];
    sum -= removed;
    printf("\tDequeued element : %d (Current Sum = %d)\n", removed,
sum);
}

```

```

void display()
{
    if (front == -1 || front > rear)
    {
        printf("\tQueue Empty!!\n");
        return;
    }
    printf("\tQueue Contains : ");

```

```

    for (int i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }
    printf("\n");
}

int main()
{
    int choice, item;
    //char cont = 'y';
    printf("----- MENU ----- \n");

    printf("\t1. Enqueue\n\t2. Dequeue\n\t3. Display\n\t4. Exit\n");
    printf("\tWhat type of operation do you want to perform?\n");

    while (1)
    {
        //system("cls");

        printf("\tEnter your choice : ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("\t> Your choice : Enqueue operation\n");
                printf("\tEnter a number to enqueue : ");
                scanf("%d", &item);
                enqueue(item);

```

```

        break;

    case 2:
        printf("\t> Your choice : Dequeue operation\n");
        dequeue();
        break;

    case 3:
        printf("\t> Your choice: Display operation\n");
        display();
        break;

    case 4:
        printf("\tBYE!!\n");
        exit(0);

    default:
        printf("\tInvalid Input!!\n");
}

//printf("\n\tDo you want to continue? (y for yes/any other key for no)
: ");
//scanf(" %c", &cont);
}

printf("\n\tProgram Terminated. Goodbye!!\n");
return 0;
}

```

"C:\Users\Iconic Galaxy\Desktop\code practice\CSE 207\R\Untitled7.exe"

1. Enqueue
2. Dequeue
3. Display
4. Exit

What type of operation do you want to perform?

Enter your choice : 1

> Your choice : Enqueue operation

Enter a number to enqueue : 20

Element 20 inserted to Queue.(Current Sum = 20)

Enter your choice : 1

> Your choice : Enqueue operation

Enter a number to enqueue : 15

Element 15 inserted to Queue.(Current Sum = 35)

Enter your choice : 1

> Your choice : Enqueue operation

Enter a number to enqueue : 18

Sum limit exceeded. Cannot insert 18 (Current Sum = 35)

Enter your choice : 1

> Your choice : Enqueue operation

Enter a number to enqueue : 10

Element 10 inserted to Queue.(Current Sum = 45)

Enter your choice : 3

> Your choice: Display operation

Queue Contains : 20 15 10

Enter your choice : 4

BYE!!

Process returned 0 (0x0) execution time : 33.959 s

Press any key to continue.